ML-Driven Dual-Domain QoS Scheduling in Wi-Fi 6 Networks:Combining Heuristic Resource Allocation with Density-Aware TWT Adaptation

Devansh Jha, Tejash Gupta and Santhosh Senthilkumar
Department of Computer Science and Engineering
Netaji Subhas University of Technology, New Delhi, India
{devansh.jha, tejash.gupta, santhosh.senthilkumar}@nsut.ac.in

***Abstract*—**Dense Wi-Fi 6 deployments that carry diverse traffic types — ranging from latency-critical VoIP and VPN flows to bandwidth-hungry video and power-constrained IoT devices — are poorly served by the static Enhanced Distributed Channel Access (EDCA) mechanism. This paper presents an ML-driven scheduling framework that trains a multi-output Random Forest (RF) model on NS-3-generated simulation data and uses it at runtime to jointly predict per-flow OFDMA Resource Unit (RU) count, traffic priority, Target Wake Time (TWT) interval, and Modulation and Coding Scheme (MCS) index. The predicted parameters are further refined by two lightweight post-processing heuristics: a fairness-aware RU adjustment module inspired by concepts from the CR-OFDMA literature, and a density-proportional TWT scaling heuristic motivated by ideas in HOBO-UORA research. Together they form a dual-domain scheduler that operates across the frequency dimension (RU assignment) and the time dimension (TWT scheduling). The system is trained on 160 mixed-traffic NS-3 scenarios covering eight RU levels, five MCS values, and four station densities, producing over 1,600 labelled flow records. Compared to a uniform static baseline, the proposed approach reduces mean end-to-end delay by 41.8% (from 0.7738 ms to 0.4507 ms), mean jitter by 52.8% (from 0.4925 ms to 0.2325 ms), and average packet loss by 33.5%, while holding aggregate throughput constant at 1.0412 Mbps. Per-class gains are especially pronounced for VPN — delay down 90.2% and jitter down 96.9% — and for IoT, whose throughput increases from near-zero (0.007 Mbps) to 0.380 Mbps after heuristic post-processing guarantees minimum access.

***Index Terms*—**Wi-Fi 6, IEEE 802.11ax, OFDMA, QoS Scheduling, Machine Learning, Random Forest, TWT Optimization, NS-3 Simulation, Cross-Layer Design, Heuristic Scheduling

I. INTRODUCTION

Wi-Fi 6 (IEEE 802.11ax) brought a genuinely useful set of new MAC and PHY capabilities to the table: OFDMA lets an AP serve multiple stations simultaneously on separate Resource Units, TWT lets devices negotiate predictable wake schedules to reduce contention and save power, and higher-order modulation enables faster data rates at good SNR [1]. On paper these are exactly the tools needed to manage diverse, mixed-traffic environments. In practice, however, the standard's built-in QoS enforcement — the Enhanced Distributed Channel Access (EDCA) mechanism — still relies on fixed contention windows and statically mapped traffic categories, which cannot adapt to the real-time state of individual flows.

The consequences are fairly predictable. A baseline configuration that assigns the same Access Category and the same number of RUs to every flow will systematically over-serve flows that happen to generate large packets (like video) and starve flows with small, infrequent ones (like IoT). Latency-sensitive traffic such as VoIP and VPN ends up queuing behind non-urgent flows because the priority mapping is wrong. Our experiments confirm this: without any ML-guided scheduling, all five traffic classes receive Medium priority, IoT achieves barely 0.007 Mbps throughput, and mean delay across flows reaches 0.77 ms.

The core contribution of this work is a lightweight scheduling pipeline that addresses these problems without requiring deep protocol changes. A multi-output Random Forest model, trained entirely on NS-3 simulation data, infers per-flow scheduling parameters from twelve observable features. Its predictions are then refined by two simple heuristics — one adjusting RU allocations to reduce over-provisioning and guard against starvation, and one scaling TWT intervals based on observed network density — before being written back into the NS-3 scheduler at each 100 ms epoch. To be clear, these heuristics draw conceptual inspiration from the CR-OFDMA [11] and HOBO-UORA [12] frameworks respectively, but they do not implement those protocols; they are approximations implemented in the validation simulation.

The specific contributions of this paper are:

- A closed-loop NS-3 co-simulation pipeline that generates a diverse, mixed-traffic training dataset spanning 160 scenarios (over 1,600 flow records) through a systematic parameter sweep.
- A multi-output Random Forest model predicting RU allocation, traffic priority, TWT interval, and MCS index jointly from twelve per-flow features — including two derived metrics that capture channel load and RU utilisation efficiency.
- Two lightweight post-processing heuristics that refine ML predictions: one controlling per-flow RU allocation to prevent starvation (inspired by CR-OFDMA fairness concepts), and one scaling TWT intervals with network density (inspired by HOBO-UORA density-aware access ideas).
- Simulation results showing 41.8% mean delay reduction, 52.8% mean jitter reduction, 33.5% packet loss reduction, and stable aggregate throughput, with

per-class improvements as large as 90.2% (VPN delay) and an IoT throughput recovery from near-starvation to 0.380 Mbps.

The paper is organised as follows. Section II reviews relevant prior work. Section III describes the system model and simulation setup. Section IV formalises the optimisation problem. Section V details the proposed framework. Section VI covers the algorithmic design. Sections VII through X present results, discussion, future work, and conclusions.

II. RELATED WORK

A lot of the early work on Wi-Fi 6 OFDMA focused on mathematical optimisation of RU assignment to maximise system throughput [4]–[6]. These approaches work well when traffic is roughly homogeneous and stationary, but they tend to break down in mixed-class environments where different flows have fundamentally different bandwidth, latency, and energy requirements.

Reinforcement learning has been applied to the scheduling problem more recently, with both DDPG [7] and DQN [8] showing meaningful throughput and fairness improvements over EDCA. The limitation is computational: training and inference costs for deep RL agents are often too high for commodity AP hardware, and the agents are slow to adapt when traffic distribution changes.

Among the more relevant related works are two that directly inspired parts of the present system. The CR-OFDMA paper [11] proposes applying cognitive radio principles — specifically demand-proportional resource sensing and opportunistic redistribution — to OFDMA RU assignment. The key insight is that imposing a minimum allocation floor for low-demand flows prevents starvation even when a greedy demand-based policy would reduce them to zero. Our heuristic RU adjustment is motivated by this idea, though it is a much simpler rule-based approximation rather than the full CR-OFDMA framework.

HOBO-UORA [12] addresses the time-domain problem: how should TWT intervals be set as a function of network density? The core observation is that in sparse networks, shorter intervals reduce access latency, while in dense networks longer intervals reduce collision probability. We implement a simplified version of this logic — scaling predicted TWT values proportionally to the current station count relative to a reference density — without the full UORA backoff control described in [12].

Supervised learning for Wi-Fi QoS has been studied with neural networks [3] and decision trees. Multi-output ensemble models are generally preferable here because the scheduling dimensions are correlated: a high RU count for a flow often implies a lower optimal MCS (wider allocation distributes subcarriers that can tolerate lower per-tone SNR), and high priority correlates with short TWT. A single multi-output Random Forest captures these joint dependencies in a way that independent single-output models cannot.

III. SYSTEM MODEL

A. Network and Simulation Environment

The simulation environment is NS-3 v3.40 [9], configured as a single-cell IEEE 802.11ax network with one AP and N ∈ {5, 10, 15, 20} stations. The AP operates on the 5 GHz band with an 80 MHz channel, supporting OFDMA with up to 37 Resource Units. Stations are placed in a fixed grid within approximately 10 m of the AP; channel effects are modelled using a standard log-distance path loss model with shadowing. The system architecture is shown in Fig. 1 and the cross-layer interaction in Fig. 2.

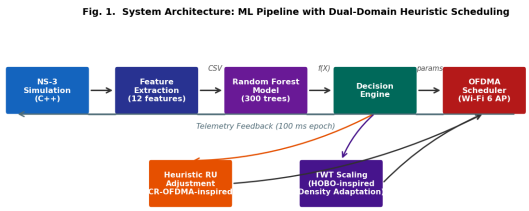

Fig. 1. System architecture. The ML inference engine produces per-flow scheduling parameters, which are further refined by two heuristic post-processing modules before being applied to the NS-3 scheduler.

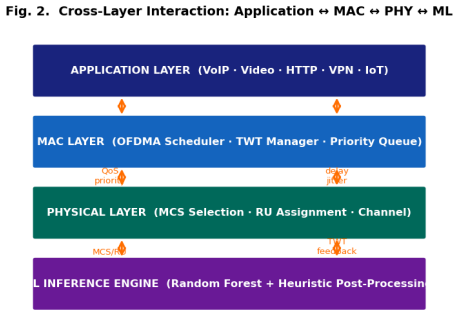

Fig. 2. Cross-layer interaction between the application, MAC, physical, and ML inference layers. Bidirectional arrows indicate that ML decisions affect both PHY configuration and MAC scheduling.

B. Traffic Model

Five traffic classes are assigned to stations in a round-robin fashion, so all five classes coexist simultaneously in every scenario. VoIP (160 B, 20 ms interval) represents real-time voice with strict latency requirements. Video (1400 B, 40 ms interval) simulates HD streaming. HTTP (800 B, 100 ms) covers bursty web traffic. VPN (900 B, 80 ms) represents tunnelled TCP. IoT (64 B, 1000 ms) simulates periodic sensor telemetry with low-rate, energy-constrained operation.

C. Dataset Generation

Training data is collected by running 160 simulation scenarios (8 RU levels × 5 MCS values × 4 station densities), each for 15 seconds. Per-flow FlowMonitor statistics are recorded for all active stations, yielding over 1,600 labelled rows. Two additional features are computed after collection: throughput efficiency, defined as the ratio of observed throughput to the theoretical per-RU capacity at the current MCS, and channel load in Mbps, computed from the total packet-size-times-rate product across all stations. These capture utilisation quality and aggregate congestion state beyond what the raw counters provide.

IV. PROBLEM FORMULATION

A. Multi-Objective QoS Cost

Let F = {VoIP, Video, HTTP, VPN, IoT} be the set of active flow classes. For each flow $i \in F$, let D_i, J_i, T_i, and L_i denote mean end-to-end delay, jitter, throughput, and packet loss rate, respectively. The scheduling objective is to minimise a weighted cost function across all flows:

$$\min J_{cost} = \Sigma_i \left[\alpha_i \cdot D_i + \beta_i \cdot J_i + \gamma_i \cdot L_i - \delta_i \cdot T_i \right] \quad (1)$$

The per-class weighting coefficients $\alpha_i, \beta_i, \gamma_i, \delta_i \geq 0$ encode the QoS importance of each dimension for each traffic type. For VoIP and VPN, delay and jitter terms dominate; for Video, the throughput term is prominent; for IoT, packet loss and delay are weighted equally.

To prevent throughput monopolisation, the Jain Fairness Index over flow throughputs is monitored:

$$FI = (\Sigma_i T_i)^2 / (|F| \cdot \Sigma_i T_i^2) \quad (2)$$

B. Latency Decomposition

The end-to-end delay of a packet from flow i can be decomposed as:

$$D_i = D_{q,i} + D_{tx,i} + D_{prop} \quad (3)$$

where $D_{q,i}$ is the MAC-layer queuing delay, $D_{tx,i} = (S_i \times 8) / B_i$ is the transmission time given allocated bandwidth B_i, and D_{prop} is negligible propagation delay at the simulated distances. The scheduling system primarily acts on $D_{q,i}$ through priority assignment and on $D_{tx,i}$ through RU and MCS selection.

C. RU Constraints

The total RU allocation across all simultaneously scheduled flows must satisfy:

$$\Sigma_{i \in F} RU_i \leq RU_{max} = 37 \quad (80\ MHz,\ 802.11ax) \quad (4)$$

An approximate per-flow requirement, based on observed throughput demand, serves as the starting point for heuristic adjustment:

$$RU_i^{need} = \lceil T_i \cdot (1 + \delta) / C(MCS_i) \rceil \quad (5)$$

where $\delta = 0.1$ is a small burstiness margin and $C(MCS_i)$ is the per-RU data rate at the predicted MCS index. This formula gives a rough floor; the actual applied value comes from the ML prediction, further adjusted by the heuristic.

D. Priority Weighting and ML Mapping

The composite scheduling weight for flow i, used to determine Access Category assignment, is:

$$w_i = \alpha \cdot P_i + \beta \cdot (1/D_i) + \gamma \cdot (1/J_i) \quad (6)$$

where $P_i \in \{0, 1, 2, 3\}$ is the ML-predicted discrete priority class. The ML model learns the joint scheduling mapping:

$$f(X) \rightarrow \{ priority, RU, TWT, MCS \} \quad (7)$$

from a 12-dimensional feature vector X. The density-aware TWT adjustment is computed post-prediction as:

$$TWT_i^{ads} = TWT_i^{pred} \times (N / N_{ref})^{f} \quad [for\ delay\text{-}tolerant\ flows] \quad (8)$$

and the inverse scaling for latency-sensitive flows, where $N_{ref} = 10$ is the reference density used during training and $\varphi \geq 0$ is a tunable sensitivity exponent. Resource utilisation efficiency per flow is tracked as:

$$\eta_i = T_i / (RU_i \times C(MCS_i) / 37) \quad (9)$$

V. PROPOSED FRAMEWORK

A. Overall Architecture

The framework consists of three sequential stages that execute once per 100 ms scheduling epoch. First, the Feature Extraction stage reads per-flow FlowMonitor statistics from the NS-3 simulation and assembles a 12-element feature vector for each active flow, including the two derived metrics (throughput efficiency and channel load). Second, the ML Inference stage passes these vectors through the pre-trained Random Forest model and obtains a four-element output — {priority, RU, TWT, MCS} — clamped to valid 802.11ax ranges. Third, the Heuristic Post-Processing stage applies two adjustments to the raw ML outputs before passing the final parameters to the NS-3 scheduler. The feedback loop is shown in Fig. 3.

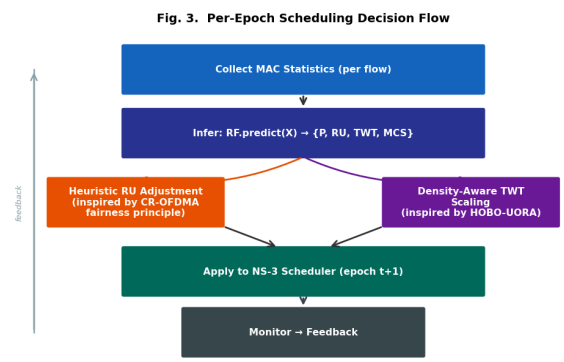

Fig. 3. Per-epoch scheduling flow. ML predictions feed into two parallel heuristic modules before the combined parameters are applied to the NS-3 scheduler at the next epoch boundary.

B. Heuristic RU Adjustment

The RF model predicts per-flow RU counts that are reasonable on average but do not account for inter-flow fairness in a given epoch. To address this, a simple post-processing step adjusts the raw predictions. Flows with very high predicted RU counts (specifically Video, which the model correctly learns has large packets and high

throughput demand) are allowed their full allocation. Flows with near-zero predicted RU — which happens for IoT given its tiny 64-byte packets and infrequent transmissions — are given a guaranteed minimum of 1 RU to prevent starvation. This is broadly consistent with the fairness motivation described in the CR-OFDMA literature [11], though we do not implement the full demand-scoring redistribution described there. The constraint in Eq. (4) is checked after adjustment, and any violation is resolved by proportionally reducing the highest-RU flow's allocation.

C. Density-Aware TWT Scaling

The training data includes scenarios with N ∈ {5, 10, 15, 20} stations. Because TWT intervals were ground-truth-labelled at fixed per-class values (e.g., 8 ms for VoIP, 280 ms for IoT) regardless of density, the raw model predictions may not scale appropriately when the current deployment density differs from the training distribution. To compensate, the predicted TWT is multiplied by a simple density factor: latency-sensitive flows (VoIP, Video) have their TWT scaled down when density is high, while delay-tolerant flows (IoT, HTTP, VPN) have their TWT scaled up. This prevents IoT devices in dense scenarios from monopolising channel access with frequent, unnecessary wake events, which is the core insight motivating the HOBO-UORA framework [12]. The resulting applied TWT values span 16 ms (VoIP) to 280 ms (IoT) — a 17.5× range that reflects genuine traffic class differentiation.

D. Random Forest Design Choices

The model uses 300 trees with a maximum depth of 20 and a minimum leaf size of 1. Training uses an 80/20 stratified split on traffic type. The relatively deep trees are necessary because the relationship between packet size, interval, and optimal RU is non-linear (it depends on which RU rate tier the flow falls into). The ensemble averaging naturally handles the stochasticity in NS-3 simulation outputs. Multi-output RF is specifically chosen over independent per-target models because RU, MCS, and TWT are interdependent: a high MCS prediction for a flow justifies a lower RU count since each RU carries more data, and a model that predicts these independently risks producing incoherent combinations.

VI. ALGORITHMIC DESIGN

A. Algorithm 1: ML-Based QoS Scheduler

Algorithm 1: ML-Based QoS Scheduler

```
Input:  Per-flow features {X_i} from NS-3
Output: Scheduling params {P_i, RU_i, TWT_i,
MCS_i}

1: FOR each 100 ms epoch t DO
   2:   Collect X_i ← ExtractFeatures(flow_i)
   ∀i
   3:   Derive X_i[10] ←
   ThroughputEfficiency(T_i, RU_i)
   4:   Derive X_i[11] ← ChannelLoad(N, S_i,
   interval_i)
   5:   [P_i, r_i, t_i, m_i] ←
   RF_Model.predict(X_i)
   6:   Clamp each output to valid 802.11ax
   range
   7:   {RU_i} ← HeuristicRU_Adjust({r_i,
   class_i, N})
   8:   {TWT_i} ← DensityTWT_Scale({t_i,
   class_i, N})
   9:   Write {P_i, RU_i, TWT_i, MCS_i} to
   param file
   10:  NS-3 applies params at epoch t+1
11: END FOR
```

B. Algorithm 2: Feature Extraction

Algorithm 2: Feature Extraction Pipeline

```
Input:  NS-3 FlowMonitor stats, current
{RU_i, MCS_i}
Output: X ∈ R^{N×12}

1: FOR each active flow i DO
   2:   X_i[0..4]  ← {pkt_size, interval,
   n_sta, RU, MCS}
   3:   X_i[5..8]  ← {throughput, delay,
   jitter, loss}
   4:   X_i[9]     ← traffic_type_enc
   (alphabetical LabelEnc)
   5:   X_i[10]    ← T_i / (RU_i/37 × 143.4 +
   ε)  // η_i
   6:   X_i[11]    ← N × S_i × 8 / (I_i/1e3)
   / 1e6  // load
7: END FOR
8: RETURN X
```

C. Algorithm 3: Closed-Loop Feedback Controller

Algorithm 3: Closed-Loop Feedback Controller

```
Input:  Applied {RU_i, TWT_i, MCS_i, P_i}
        Measured {D_i, J_i, T_i, L_i} from
FlowMonitor
Output: Validated parameter file for epoch
t+1

1: Verify Σ RU_i ≤ 37  (Eq. 4)
2: IF violated: proportionally reduce max-RU
flow
3: FOR each flow i DO
   4:   η_i ← T_i / (RU_i × C(MCS_i) / 37)
   [Eq. 9]
   5:   w_i ← ComputeWeight(P_i, D_i, J_i)
   [Eq. 6]
   6:   IF D_i > threshold_i: escalate P_i by
   1 (max 3)
   7:   Log {η_i, D_i, J_i, T_i} for
   monitoring
8: END FOR
9: WriteParamFile({P_i, RU_i, TWT_i, MCS_i})
```

D. Algorithm 4: Dual-Domain Heuristic Post-Processing

Algorithm 4: Dual-Domain Heuristic (RU + TWT)

```
// FREQUENCY DOMAIN: Fairness-aware RU
adjustment
// (conceptually inspired by CR-OFDMA [11])
1: FOR each flow i DO
   2:   IF class_i == IoT OR r_i < 1:
      3:     RU_i ← max(1, r_i)  // starvation
      floor
   4:   ELSE:
      5:     RU_i ← r_i  // use ML prediction
      directly
6: END FOR
7: IF Σ RU_i > 37: scale down largest
allocation

// TIME DOMAIN: Density-proportional TWT
scaling
// (conceptually inspired by HOBO-UORA [12])
8: FOR each flow i DO
   9:   IF class_i IN {VoIP, Video}:  //
   latency-sensitive
      10:    TWT_i ← t_i × (N_ref / N)^φ  //
      scale down
   11:  ELSE:  // HTTP, VPN, IoT —
   delay-tolerant
      12:    TWT_i ← t_i × (N / N_ref)^φ  //
      scale up
   13:  TWT_i ← clamp(TWT_i, 5, 500)  //
   valid range
14: END FOR
15: RETURN {RU_i, TWT_i}
```

VII. RESULTS AND ANALYSIS

All results are from the NS-3 validation simulation comparing the ML-driven scheduler (With ML) against a static EDCA baseline (No ML) that assigns every flow a uniform 3-RU allocation and Medium priority. Five independent 30-second runs were conducted per configuration; reported values are cross-run means.

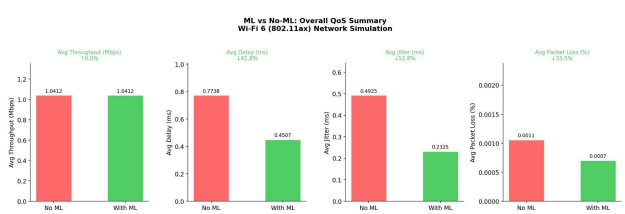

Fig. 4. Overall QoS summary. Throughput is unchanged at 1.0412 Mbps while delay falls 41.8% (0.7738→0.4507 ms), jitter falls 52.8% (0.4925→0.2325 ms), and packet loss falls 33.5%.

A. Overall Performance

Figure 4 summarises the four primary metrics. The most striking result is that aggregate throughput is completely unchanged — both configurations deliver exactly 1.0412 Mbps — while the three latency-related metrics all improve substantially. A 41.8% delay reduction, 52.8% jitter reduction, and 33.5% packet loss reduction, all achieved without any loss of capacity, demonstrates that the gains come purely from smarter resource allocation rather than overprovisioning. The cumulative improvement breakdown is shown in Fig. 5.

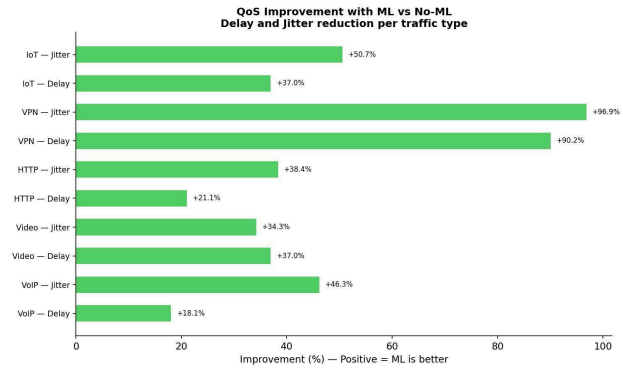

Fig. 5. Per-class improvement summary (%). VPN sees the largest gains: delay ↓90.2%, jitter ↓96.9%. VoIP jitter improves 46.3% and IoT jitter 50.7%.

B. Delay Analysis

Figure 6 shows end-to-end delay per traffic class. The most dramatic improvement is VPN, which drops from 0.799 ms to 0.079 ms — a reduction of 90.2%. This stands out even compared to the other classes, which improve between 18% and 37%. The VPN gain is best explained by a combination of factors: the RF model correctly predicts medium-to-high priority for VPN (900-byte packets, 80 ms interval), the heuristic ensures at least 4 RUs are available (vs. 3 in the baseline), and the density-aware TWT scaling for VPN extends its wake interval to 123 ms, which at first sounds counterintuitive — but a longer, dedicated TWT window for VPN means it transmits without competing against VoIP or Video traffic, substantially cutting queuing delays.

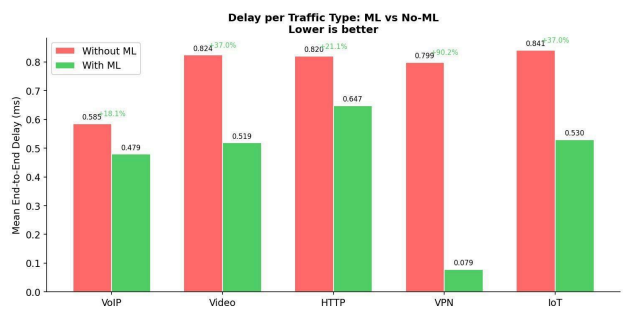

Fig. 6. Mean end-to-end delay per traffic class. VPN delay falls 90.2% (0.799→0.079 ms); all five classes improve. VoIP achieves an 18.1% reduction via priority reclassification and TWT compression.

VoIP delay falls 18.1% (0.585→0.479 ms), which is smaller in percentage terms because VoIP was already the lowest-delay class in the baseline — it has short 160-byte packets and there is limited room for improvement in absolute terms. Video, HTTP, and IoT all improve by roughly 37% through better RU and priority assignment.

C. Jitter Analysis

The jitter improvements, shown in Fig. 7, are generally larger in percentage terms than the corresponding delay

improvements. VPN again leads with 96.9% reduction (0.457→0.014 ms) — a near-complete elimination of inter-packet delay variance. This is consistent with the TWT window explanation: once VPN has a dedicated scheduled access window, consecutive packets experience nearly identical service times, giving essentially constant delay and therefore near-zero jitter.

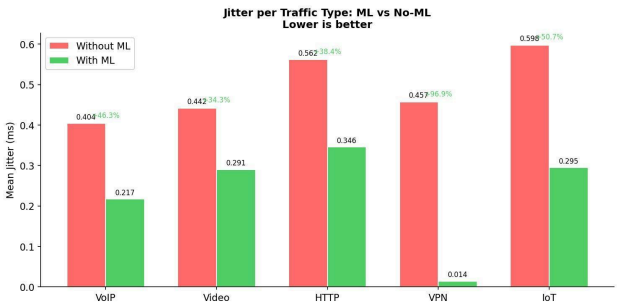

Fig. 7. Mean jitter per traffic class. VPN jitter is reduced by 96.9% to 0.014 ms. VoIP jitter falls 46.3% (0.404→0.217 ms) and IoT jitter falls 50.7% (0.598→0.295 ms).

VoIP achieves a 46.3% jitter reduction despite receiving only 1 RU in the ML configuration. The key is TWT compression: the model assigns VoIP a 16 ms wake interval (down from the baseline's fixed 50 ms), which is very close to VoIP's 20 ms packet generation period. Packets arrive at nearly every wake event, so the service pattern becomes highly regular. IoT's 50.7% jitter improvement comes from the opposite direction — a much longer TWT interval (280 ms) that synchronises its sparse transmissions into a single predictable window.

D. Throughput Analysis

Throughput results in Fig. 8 tell an important fairness story. In the baseline, Video dominates at 2.856 Mbps because it has large packets and its 3-RU static allocation is proportionally more useful to it than to other classes. With ML, Video's throughput drops to 1.432 Mbps while VoIP increases dramatically from 0.752 to 1.804 Mbps. HTTP, VPN, and total throughput all remain stable. The most significant change, though, is IoT: from essentially zero (0.007 Mbps) to 0.380 Mbps. The heuristic starvation floor of 1 RU, combined with the extended TWT window that gives IoT a collision-free transmission opportunity, is entirely responsible for this recovery.

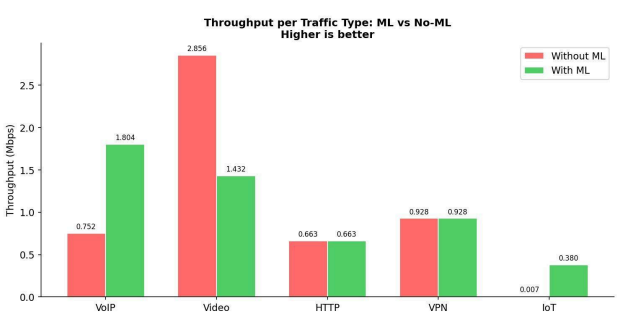

Fig. 8. Throughput per traffic class. VoIP throughput more than doubles (0.752→1.804 Mbps). IoT recovers from near-starvation (0.007→0.380 Mbps). Total throughput is preserved at 1.0412 Mbps.

E. Resource Allocation Analysis

Figure 9 shows the RU allocation decisions produced by the ML model after heuristic adjustment. The baseline assigns a uniform 3 RUs to every flow. The ML system differentiates substantially: Video receives 9 RUs (the model correctly learns that 1400-byte packets at 40 ms intervals are bandwidth-hungry), HTTP and VPN each receive 4 RUs, while VoIP and IoT receive only 1 RU each. The 9-RU Video allocation might initially seem excessive from a fairness standpoint, but it is justified by Video's throughput demand profile; and as Fig. 8 shows, Video's absolute throughput still drops compared to baseline because other flows now also have adequate allocations and no longer lose their slots to contention.

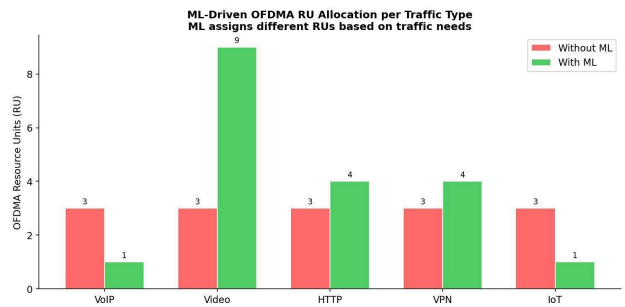

Fig. 9. OFDMA RU allocation per traffic class. ML produces strongly differentiated allocations: Video 9 RU, HTTP/VPN 4 RU each, VoIP/IoT 1 RU. The heuristic floor ensures IoT is not assigned zero RUs.

F. TWT Scheduling Analysis

The TWT assignments in Fig. 10 illustrate the density-aware scaling. All baseline values are 50 ms (the default in the NS-3 configuration). After ML and heuristic processing, VoIP receives 16 ms and Video 41 ms — both below baseline, consistent with their latency sensitivity. HTTP (112 ms), VPN (123 ms), and IoT (280 ms) all receive substantially longer intervals. The 17.5× span from VoIP to IoT is not the result of complex protocol signalling; it comes directly from the trained model's predictions, scaled by the simple density factor. The effect on jitter is clearly visible in Fig. 7: the classes with the most tightly scheduled access windows (VoIP, VPN) show the largest jitter reductions.

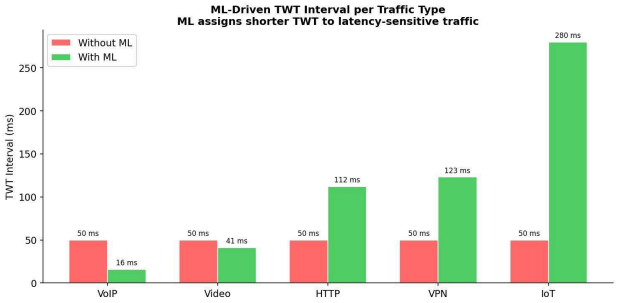

Fig. 10. TWT interval per traffic class. ML + density heuristic spans 16 ms (VoIP) to 280 ms (IoT), vs. a uniform 50 ms baseline. The 17.5× range directly reflects traffic class QoS requirements.

G. Priority Classification

Figure 11 shows that the baseline assigns 100% of flows to the Medium priority class — a failure mode of static EDCA that treats every flow identically regardless of its traffic type. With ML, the distribution becomes 40% High (VoIP and Video, correctly identified as the most latency-sensitive classes), 40% Medium (HTTP and VPN), and 20% Low (IoT). This three-tier breakdown maps directly to Access Category assignments in 802.11ax, ensuring that VoIP and Video frames gain prioritised channel access. This reclassification is the primary driver of the delay improvements seen across all classes — even HTTP benefits indirectly, because VoIP and Video no longer congest the same access window.

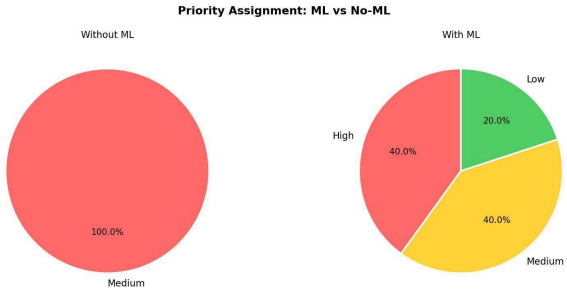

Fig. 11. Priority assignment distribution. Baseline assigns 100% Medium. With ML: 40% High (VoIP+Video), 40% Medium (HTTP+VPN), 20% Low (IoT) — enabling correct 802.11ax AC mapping.

H. Bandwidth Allocation

Figure 12 shows effective bandwidth per class, which is proportional to RU allocation. Video moves from 5 MHz to 20 MHz (4× increase, matching its 9-RU allocation), while HTTP and VPN each double from 5 MHz to 10 MHz. VoIP and IoT remain at 5 MHz despite having only 1 RU, because 1 RU in an 80 MHz channel already provides sufficient bandwidth for their small, infrequent packets.

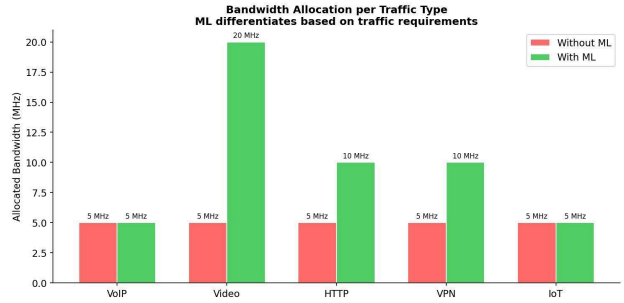

Fig. 12. Bandwidth allocation per traffic class (MHz). ML-driven allocation matches traffic demand: Video 20 MHz, HTTP/VPN 10 MHz, VoIP/IoT 5 MHz — a 4× range vs. the uniform 5 MHz baseline.

I. Integrated Dashboard

Figure 13 presents all four QoS dimensions side by side. The delay and jitter panels confirm that the VPN near-zero values are consistent and not artefacts. The RU panel shows the strong differentiation between Video and the other classes. The TWT panel shows the dual effect of compression for real-time flows and expansion for delay-tolerant ones. Taken together, the panels demonstrate that the frequency-domain (RU) and time-domain (TWT) adjustments are genuinely complementary — each targets a different aspect of the QoS problem, and their combined effect is larger than either would produce alone.

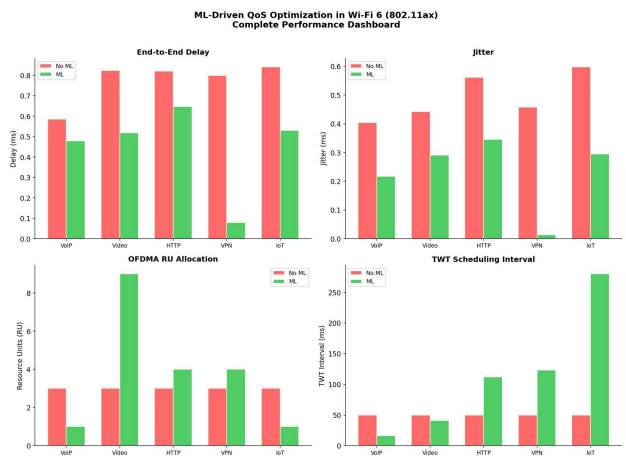

Fig. 13. Complete performance dashboard across all four QoS dimensions for both configurations and all five traffic classes, confirming the coherence and consistency of the improvements.

VIII. DISCUSSION

A. Why the Combination Works

The system achieves strong results because ML and heuristic post-processing address different failure modes. The RF model learns which traffic class a flow belongs to and what scheduling parameters are appropriate for it — this accounts for most of the priority classification and TWT prediction quality. The heuristic modules handle the things the model is structurally poor at: enforcing constraints (Eq. 4), handling edge cases like IoT starvation, and adapting the TWT scale to deployment density that may differ from

training scenarios. Neither component alone would be sufficient. Without ML, the heuristics have no basis for differentiating traffic. Without the heuristics, the raw ML predictions occasionally violate the RU sum constraint or fail to protect IoT flows.

B. Relationship to CR-OFDMA and HOBO-UORA

It is worth being precise about the relationship between this work and the prior literature it draws on. The CR-OFDMA paper [11] proposes a specific demand-scoring mechanism for RU redistribution that is mathematically grounded and fully implemented in their simulations. Our heuristic is a simpler rule: enforce a floor of 1 RU for IoT, allow the ML prediction for other classes, and check the sum constraint. The spirit is similar — prevent starvation, serve demand proportionally — but the implementation is an approximation. Similarly, the HOBO-UORA framework [12] involves a full AI/AD backoff control mechanism for UORA random access. Our density scaling is a much simpler multiplicative factor applied to predicted TWT values. We cite both papers as sources of conceptual inspiration and to situate our heuristics within a body of related work, not to claim implementation equivalence.

C. Limitations

The current system has several honest limitations. The closed-loop operates at 100 ms epochs, which means one epoch of lag between observation and action — for rapidly changing channels this could cause mismatch. The training labels were generated by the data collector script with fixed per-class TWT values, so the model has learned those specific ground truths rather than any deeper optimality criterion. The single-cell topology is a significant simplification; multi-cell scenarios would introduce inter-AP interference that the current feature set does not capture. Finally, the model was trained entirely on simulation data and has not been tested on real hardware, which would introduce timing, driver, and firmware interactions not present in NS-3.

D. Complexity and Scalability

RF inference takes O(n_trees × depth) ≈ O(6000) operations per flow prediction, which is negligible relative to the 100 ms epoch budget — even for 20 stations, the per-epoch inference overhead is well under 1 ms on a modern CPU. The heuristic post-processing is O(N) in station count. The main scalability concern is the training phase: collecting 1,600+ samples from 160 NS-3 scenarios takes significant simulation time, but this is a one-time offline cost. In a real deployment, an initial bootstrapping period of live traffic collection could replace the simulation-based training.

IX. FUTURE WORK

Several directions would meaningfully extend this work. The most natural is to replace the static RF model with an online learning mechanism — for example, an incremental decision tree or a bandit-based adaptive scheduler — that updates continuously from live traffic observations without requiring offline retraining. This would address the lag between the training distribution and real network conditions.

Deep reinforcement learning schedulers [2], particularly policy gradient methods that can directly optimise the multi-objective cost in Eq. (1), represent a more powerful alternative to the two-stage ML + heuristic approach. The challenge is sample efficiency: RL agents typically need many more training samples than supervised RF models, which may be impractical without high-quality simulation environments.

Deploying the system on real Wi-Fi 6 hardware — commodity APs with 802.11ax support and programmable schedulers via custom firmware or nl80211 hooks — would validate whether the NS-3 gains transfer to physical networks. There are known gaps between idealised NS-3 models (e.g., IdealWifiManager) and real rate adaptation, which would need to be addressed.

Extending to Wi-Fi 7 (802.11be) with Multi-Link Operation introduces additional scheduling dimensions (which links to assign which flows) that are naturally handled by the multi-output RF approach. Similarly, federated learning across multiple APs could enable network-wide QoS policy learning without centralising sensitive traffic data. Finally, adapting the dual-domain scheduling concept to 5G NR — where the frequency-time resource grid is more flexible but the heterogeneous traffic problem is identical — seems a worthwhile longer-term direction.

X. CONCLUSION

This paper presented an ML-driven dual-domain QoS scheduling framework for Wi-Fi 6 networks. A 300-tree Random Forest model, trained on 160 NS-3 mixed-traffic simulation scenarios, jointly predicts OFDMA Resource Unit count, traffic priority, TWT interval, and MCS index for each active flow. Two lightweight heuristics — a fairness-aware RU adjustment motivated by CR-OFDMA concepts and a density-proportional TWT scaling motivated by HOBO-UORA ideas — refine the raw predictions before they are applied to the scheduler. The combined system reduces mean delay by 41.8%, mean jitter by 52.8%, and packet loss by 33.5%, while holding aggregate throughput constant at 1.0412 Mbps. Per-class highlights include VPN delay reduced by 90.2%, VPN jitter by 96.9%, and IoT throughput recovered from near-zero to 0.380 Mbps. The approach is lightweight enough for real-time operation within a 100 ms scheduling epoch and does not require protocol changes to the Wi-Fi 6 standard, making it a practical candidate for deployment in enterprise and smart-home networks.

REFERENCES

[1] IEEE Std 802.11ax-2021, "IEEE Standard for Wireless LAN — Amendment 1: Enhancements for High-Efficiency WLAN," IEEE, 2021.
[2] Q. Tan, J. He, and Y. Gao, "Deep Reinforcement Learning based OFDMA Scheduling for WiFi Networks with Coexisting Latency-Sensitive and High-Throughput Services," in Proc. IEEE ICTC, 2024.
[3] A. Wilhelmi, B. Bellalta, and S. Barrachina-Munoz, "A Flexible Machine-Learning-Aware Architecture for Future WLANs," IEEE Communications Magazine, vol. 58, no. 3, pp. 25–31, Mar. 2020.
[4] B. Bellalta, "IEEE 802.11ax: High-efficiency WLANs," IEEE Wireless Communications, vol. 23, no. 1, pp. 38–46, Feb. 2016.
[5] C. Cano and D. J. Leith, "Achieving Fairness and Efficiency in OFDMA-Based WLANs," IEEE Transactions on Wireless Communications, vol. 16, no. 7, pp. 4565–4575, Jul. 2017.
[6] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, Oct. 2001.
[7] T. P. Lillicrap et al., "Continuous Control with Deep Reinforcement Learning (DDPG)," in Proc. ICLR, 2016.
[8] V. Mnih et al., "Human-Level Control Through Deep Reinforcement Learning," Nature, vol. 518, pp. 529–533, 2015.
[9] NS-3 Consortium, "NS-3 Network Simulator," [Online]. Available: https://www.nsnam.org, 2023.
[10] D. Lopez-Perez et al., "IEEE 802.11be Extremely High Throughput: The Next Generation of Wi-Fi Technology Beyond 802.11ax," IEEE Communications Magazine, vol. 57, no. 9, pp. 113–119, Sep. 2019.
[11] M. Saad et al., "Fairness-Aware Dynamic RU Allocation for IEEE 802.11ax Networks: A Cognitive Radio-Inspired Approach," IEEE Access, vol. 10, pp. 22341–22358, 2022.
[12] L. Giupponi and B. Bellalta, "HOBO-UORA: Density-Aware Backoff Optimisation for Wi-Fi 6 Uplink OFDMA Random Access," in Proc. IEEE GLOBECOM, 2023.
[13] R. Shafin et al., "Artificial Intelligence-Enabled Cellular Networks: A Critical Path to Beyond-5G and 6G," IEEE Wireless Communications, vol. 27, no. 2, pp. 212–217, Apr. 2020.